

```
In [1]: import tensorflow as tf
        from tensorflow.keras import datasets, layers, models
        from tensorflow.keras.optimizers import Adam
        from tensorflow.keras.preprocessing.image import ImageDataGenerator
        from sklearn.model_selection import train_test_split
        import matplotlib.pyplot as plt
```

```
In [2]: # -----
        # Load Dataset (MNIST)
        # -----
        (train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data
```

```
In [3]: # Normalize
        train_images = train_images / 255.0
        test_images = test_images / 255.0
```

```
In [4]: # Reshape for CNN (add channel dimension)
        train_images = train_images.reshape((60000, 28, 28, 1))
        test_images = test_images.reshape((10000, 28, 28, 1))
```

```
In [5]: # Split validation data
        train_images, val_images, train_labels, val_labels = train_test_split(
            train_images, train_labels, test_size=0.1, random_state=42
        )
```

```
In [6]: # -----
        # Data Augmentation
        # -----
        datagen = ImageDataGenerator(
            rotation_range=10,
            zoom_range=0.1,
            width_shift_range=0.1,
            height_shift_range=0.1
        )
```

```
In [7]: datagen.fit(train_images)
```

```
In [8]: # -----
        # Build CNN Model (FIXED)
        # -----
        model = models.Sequential([
            layers.Input(shape=(28, 28, 1)), # ✅ Correct way

            layers.Conv2D(32, (3, 3), activation='relu'),
            layers.MaxPooling2D((2, 2)),

            layers.Conv2D(64, (3, 3), activation='relu'),
            layers.MaxPooling2D((2, 2)),

            layers.Conv2D(128, (3, 3), activation='relu'),

            layers.Flatten(),
```

```

layers.Dropout(0.5),

layers.Dense(128, activation='relu'),
layers.Dense(10, activation='softmax')
])

```

```

In [9]: # Show model
model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 26, 26, 32)	
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	
conv2d_2 (Conv2D)	(None, 3, 3, 128)	
flatten (Flatten)	(None, 1152)	
dropout (Dropout)	(None, 1152)	
dense (Dense)	(None, 128)	
dense_1 (Dense)	(None, 10)	



Total params: 241,546 (943.54 KB)
 Trainable params: 241,546 (943.54 KB)
 Non-trainable params: 0 (0.00 B)

```

In [10]: # -----
# Compile Model
# -----
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

```

In [11]: # -----
# Train Model
# -----
history = model.fit(
    datagen.flow(train_images, train_labels, batch_size=64),
    epochs=20,
    validation_data=(val_images, val_labels)
)

```

Epoch 1/20

```
C:\Users\vaibhav\anaconda3\Lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignored.
```

```
self._warn_if_super_not_called()
```

844/844 ————— 42s 46ms/step - accuracy: 0.7464 - loss: 0.7615 - val_accuracy: 0.9830 - val_loss: 0.0566
Epoch 2/20

844/844 ————— 39s 47ms/step - accuracy: 0.9561 - loss: 0.1407 - val_accuracy: 0.9857 - val_loss: 0.0464
Epoch 3/20

844/844 ————— 41s 48ms/step - accuracy: 0.9684 - loss: 0.1040 - val_accuracy: 0.9868 - val_loss: 0.0412
Epoch 4/20

844/844 ————— 41s 48ms/step - accuracy: 0.9739 - loss: 0.0847 - val_accuracy: 0.9922 - val_loss: 0.0277
Epoch 5/20

844/844 ————— 39s 46ms/step - accuracy: 0.9787 - loss: 0.0703 - val_accuracy: 0.9895 - val_loss: 0.0334
Epoch 6/20

844/844 ————— 38s 45ms/step - accuracy: 0.9801 - loss: 0.0645 - val_accuracy: 0.9910 - val_loss: 0.0312
Epoch 7/20

844/844 ————— 38s 45ms/step - accuracy: 0.9816 - loss: 0.0599 - val_accuracy: 0.9902 - val_loss: 0.0289
Epoch 8/20

844/844 ————— 38s 45ms/step - accuracy: 0.9833 - loss: 0.0564 - val_accuracy: 0.9908 - val_loss: 0.0329
Epoch 9/20

844/844 ————— 38s 45ms/step - accuracy: 0.9841 - loss: 0.0517 - val_accuracy: 0.9922 - val_loss: 0.0289
Epoch 10/20

844/844 ————— 38s 45ms/step - accuracy: 0.9857 - loss: 0.0473 - val_accuracy: 0.9932 - val_loss: 0.0240
Epoch 11/20

844/844 ————— 38s 45ms/step - accuracy: 0.9859 - loss: 0.0465 - val_accuracy: 0.9922 - val_loss: 0.0265
Epoch 12/20

844/844 ————— 39s 46ms/step - accuracy: 0.9865 - loss: 0.0453 - val_accuracy: 0.9942 - val_loss: 0.0214
Epoch 13/20

844/844 ————— 38s 44ms/step - accuracy: 0.9873 - loss: 0.0411 - val_accuracy: 0.9940 - val_loss: 0.0220
Epoch 14/20

844/844 ————— 40s 48ms/step - accuracy: 0.9881 - loss: 0.0387 - val_accuracy: 0.9947 - val_loss: 0.0217
Epoch 15/20

844/844 ————— 39s 46ms/step - accuracy: 0.9884 - loss: 0.0392 - val_accuracy: 0.9918 - val_loss: 0.0275
Epoch 16/20

844/844 ————— 39s 46ms/step - accuracy: 0.9890 - loss: 0.0372 - val_accuracy: 0.9930 - val_loss: 0.0246
Epoch 17/20

844/844 ————— 39s 46ms/step - accuracy: 0.9882 - loss: 0.0382 - val_accuracy: 0.9938 - val_loss: 0.0234
Epoch 18/20

844/844 ————— 38s 45ms/step - accuracy: 0.9891 - loss: 0.0349 - val_accuracy: 0.9950 - val_loss: 0.0201
Epoch 19/20

844/844 ————— 38s 45ms/step - accuracy: 0.9897 - loss: 0.0332 - val_accuracy: 0.9933 - val_loss: 0.0255

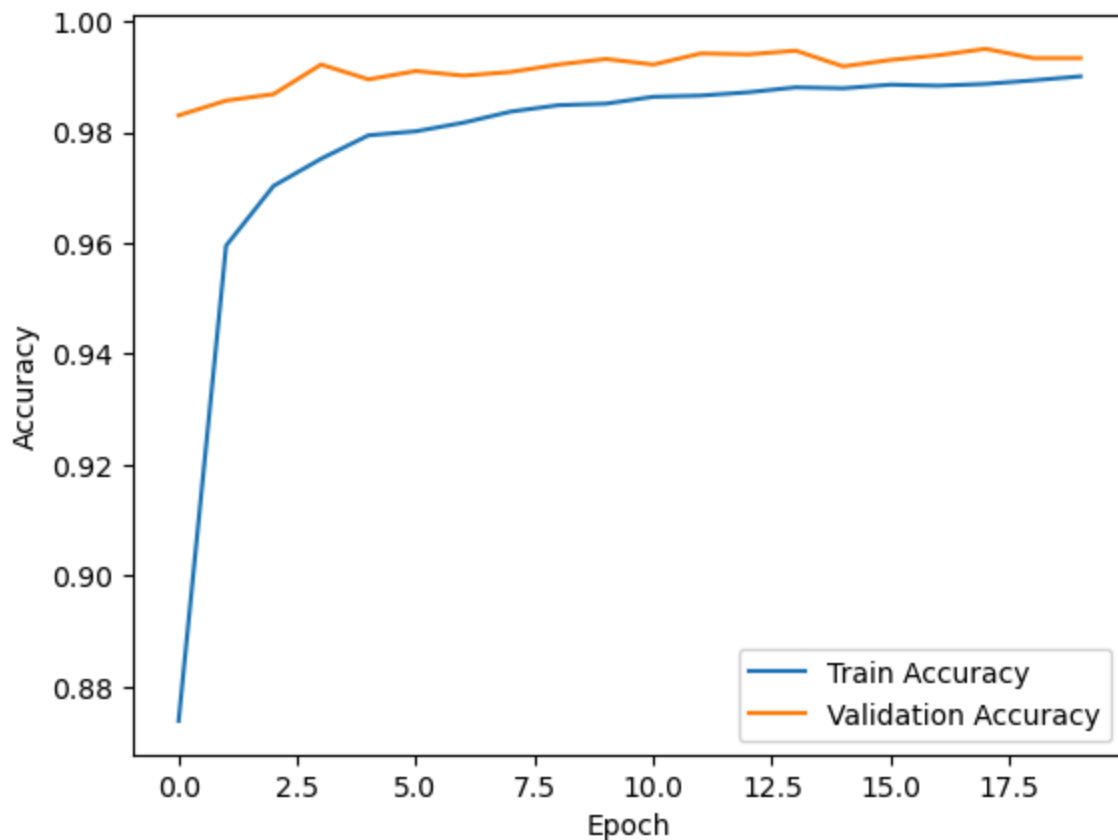
Epoch 20/20

844/844 ————— 37s 44ms/step - accuracy: 0.9903 - loss: 0.0340 - val_accuracy: 0.9933 - val_loss: 0.0288

```
In [12]: # -----  
# Evaluate Model  
# -----  
test_loss, test_acc = model.evaluate(test_images, test_labels)  
print("Test Accuracy:", test_acc)
```

313/313 ————— 2s 5ms/step - accuracy: 0.9944 - loss: 0.0186
Test Accuracy: 0.9951000213623047

```
In [26]: # Plot Accuracy  
plt.plot(history.history['accuracy'], label='Train Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```



```
In [28]: # -----  
# Prediction Example (NEW)  
# -----  
import numpy as np  
  
sample = test_images[0].reshape(1, 28, 28, 1)  
prediction = model.predict(sample)
```

```
print("Predicted Digit:", np.argmax(prediction))  
print("Actual Digit:", test_labels[0])
```

1/1  0s 274ms/step

Predicted Digit: 7

Actual Digit: 7

In []: